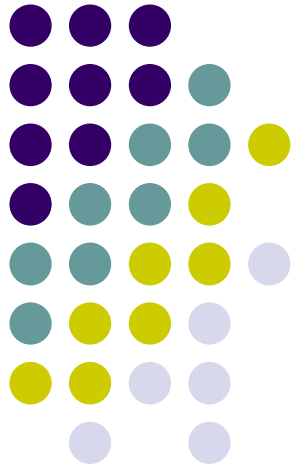
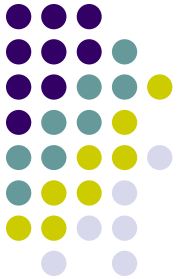
CS 420

Design of Algorithms

Analytical Models of Parallel Algorithms

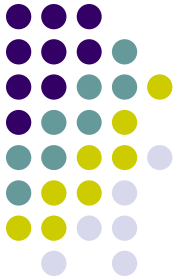


Analytical Models of Parallel Algoritms



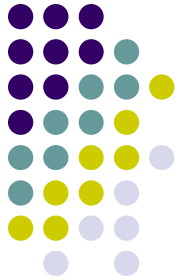
- Remember: minimize parallel overhead

Sources of Overhead



- Interprocess interactions
  - Almost any nontrivial (non-embarrassingly) parallel algorithm will require interprocess interaction.
  - This is overhead with respect to the serial algorithm to achieve the same solution
    - Remember: decomposition and mapping

Sources of Overhead



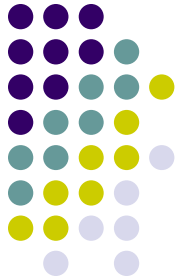
- Idling
  - Idling processes in an algorithm = net loss in aggregate computational performance.
  - i.e. not squeezing as much performance out of the paraallel algorithms as (maybe) possible
  - = overhead

Sources of Overhead



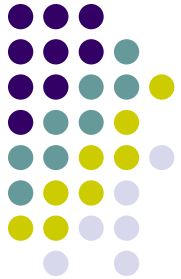
- Excess Computation
  - The best existing serial algorithm may not be readily or efficiently parallelizable
  - -perhaps, can't just evenly divide serial algorithm into p parallel pieces.
  - Each parallel task may require addition computation (relative to the corresponding work in the serial algorithm) – recall: redundant computation
  - = Excess computation = overhead

Performance Metrics



- Execution Time
  - Serial Runtime = total lapsed time (wall time) from the beginning to the end of execution for the serial program on a single PE.
  - Parallel Runtime = total lapsed time (wall time) from the beginning of the parallel computation to the end of the parallel computation.
  - Ts = Serial Runtime
  - Tp = Parallel Runtime

Performance Metrics



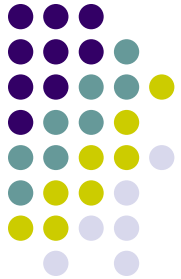
- Execution time—
  - As a baseline….**from a theoretical perspective**
  - Ts is often based on the best available serial algorithm to solution a given problem
  - … not necessarily based on the serial version of the parallel algorithm.
  - **From a practical perspective**…
  - Sometimes the serial and parallel algorithms are based on the same algorithm
  - Sometimes you want to know the parallel algorithm compares to it serial couterpart.

Performance Metrics



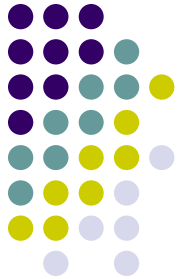
- Total Parallel Overhead
  - Need to represent the Total Parallel Overhead as an overhead function
    - Will be a function of things like work size (w) and number of PEs (p)
  - Total Parallel Overhead = total parallel runtime (Tp) * the number of PEs (p) minus the serial runtime (Ts) for the best available serial algorithm for the same problem
  - To = pTp - Ts

Performance Metrics



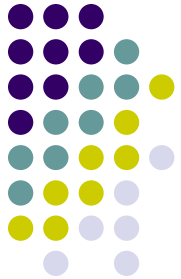
- Speedup
  - Usually we parallelize an algorithm to speed things up..
  - … therefore, the obvious question is “how much did it speed things up?”
  - Speed up = runtime of the serial algorithm (Ts) to the runtime of the parallel algorithm (Tp), or…
  - S = Ts/Tp , or
  - S= Θ(Ts/Tp)
  - For a given number of PEs (p) and given size problem

Performance Metrics



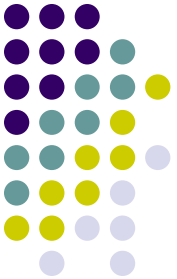
- Speedup – for example..
- Adding up n numbers with n PEs
- Serial algorithm requires n steps –
  - Communicate a number, add, communicate summ, add,…
- Parallel algorithm – even PE communicates its number to lower even neighbor, neighbor adds the numbers and passes sum …
- … binary tree

Performance Metrics



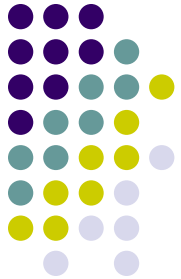
- Example adding n numbers with n PEs
  - Ts = n
  - Tp = log n
  - So..
  - S = n/log n, or
  - S = Θ(n/log n)
  - If n = 16, then
  - Ts = 16, and Tp = log 16 = 4
  - S = 16/4 = 4

Performance Metrics



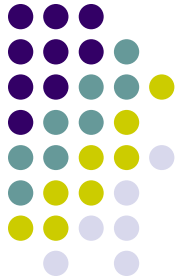
- Speedup
- In theory S can not be greater than the number of PEs (p)
- But this does occur…
- When it does in is called Superlinear speedup

Performance Metrics



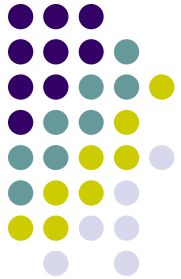
- Superlinear Speedup
- Why does this happen?
- Poor serial algorithm design
  - Maybe parallelization removed bottlenecks in the serial program
  - IO contention, for example

Performance Metrics



- Superlinear Speedup
- Cache Effects
  - Distributing a problem in smaller pieces may improve the cache hit rate and, therefore, improve the overall performance of the algorithm, more so than in proportion to the number PEs.
  - For example,….

Performance Metrics



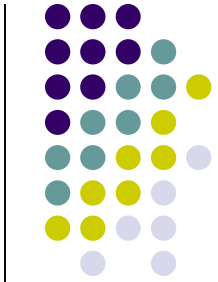
- Superlinear Speedup – Cache effects
- From A. Grama, et.al 2003
- Suppose your serial algorithm has a cache hit rate of 80%, and you have
- Cache latency of 2ns
- Memory latency of 100ns
- Then, effective memory access time is
  - 2 * 0.8 + 100 * 0.2 = 21.6ns
  - If algorithm is memory bound, one FLOP per memory access then algorithm runs at 43.6 MFLOPS

Performance Metrics



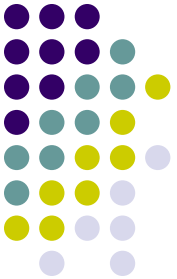
- Superlinear Speedup – Cache effects
- Now suppose you parallelize this problem on two PEs, so Wp = W/2
- Now you have remote data access to deal with, assume each remote memory access requires 400ns (much slower than direct memory and cache)
- …continued…

Performance Metrics



- Superlinear Speedup – Cache effects
- This algorithm only requires remote memory access 20% of the time
- Since Wp is smaller cache hit rate goes to 90%...
- … and local memory access is 8%
- Average memory access time =
  - 2 * 0.9 + 100 * 0.08 + 400 * 0.02 = 17.8ns
- Each PE processing rate 56.18
- Total execution rate (2 PEs) = 112.36 MFLOPS
- So…
- S = 112.36/46.3 = 2.43 (superlinear speedup)

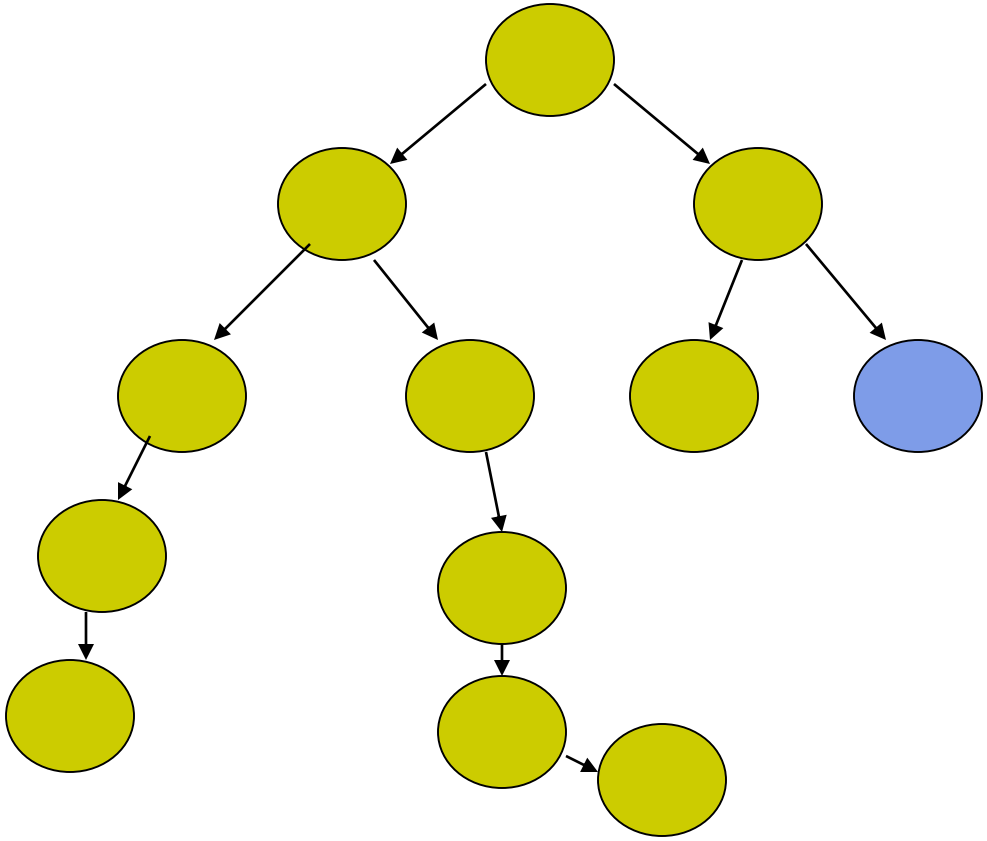
Performance Metrics



- Superlinear Speedup from Exploratory Decomposition.
- Recall that Exploratory decomposition is useful for findings solutions where the problem space is defined as a tree of alternatives… and the solution is find the correct node in the tree.

Performance Metrics

- Superlinear Speedup from Exploratory Decomposition



Blue node = solution

Use Depth-first search algorithm

Assume time to visit a node and test for solutioin = x

Serial Algorithm Ts = 12x

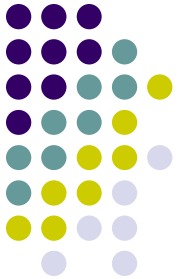
Parallel Algorithm – p=2

Parallel Algorithm Tp = 3x

S = 12/3 = 4

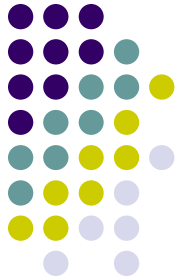
If

Performance Metrics



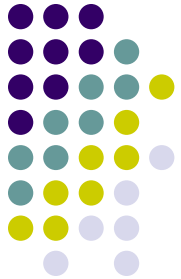
- Efficiency – a measure of how fully the algorithm utilizes processing resources
  - Ideally Speedup (S) is equal to p
  - Not typical because of overhead
  - Ideally S = p, and therefore, Efficiency (E) =1
  - Usually S < p, and 0<E<1
  - E = S/p
  - Remember: adding n numbers on n PEs
  - E = (n/log n)/n, or
  - 1/(log n)

Performance Metrics

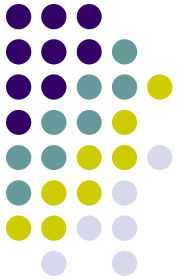


- Scalability – does the algorithm scale
  - Scalability – how well does the algorithm scale as the number of PEs scales, or…
  - How well does the algorithm scale as the size of the problem scales?
  - What does S do as you increase p?
  - What does S do as you increase w?

Perfomance Metrics



- Scalability – another way to look at it
- Scalability – can you maintain a constant E as you vary p or w.
- Is E =f(w,p)



The End

